

面向AArch64 指令集的Triton-CPU后端实现

陈正

compiler-Sig maintainer





为什么关心Triton/Triton-CPU

Triton



Triton-CPU

特点

高度抽象，硬件无关

自动并行化

.....

block-level编程模型

显示内存访问控制

多级并行抽象

统一编程模型

优势

性能接近手写CUDA

算子开发效率极高

强跨平台开发效率

极致硬件利用率

应用

FlagGems

LightLLM

.....

FlagAttention

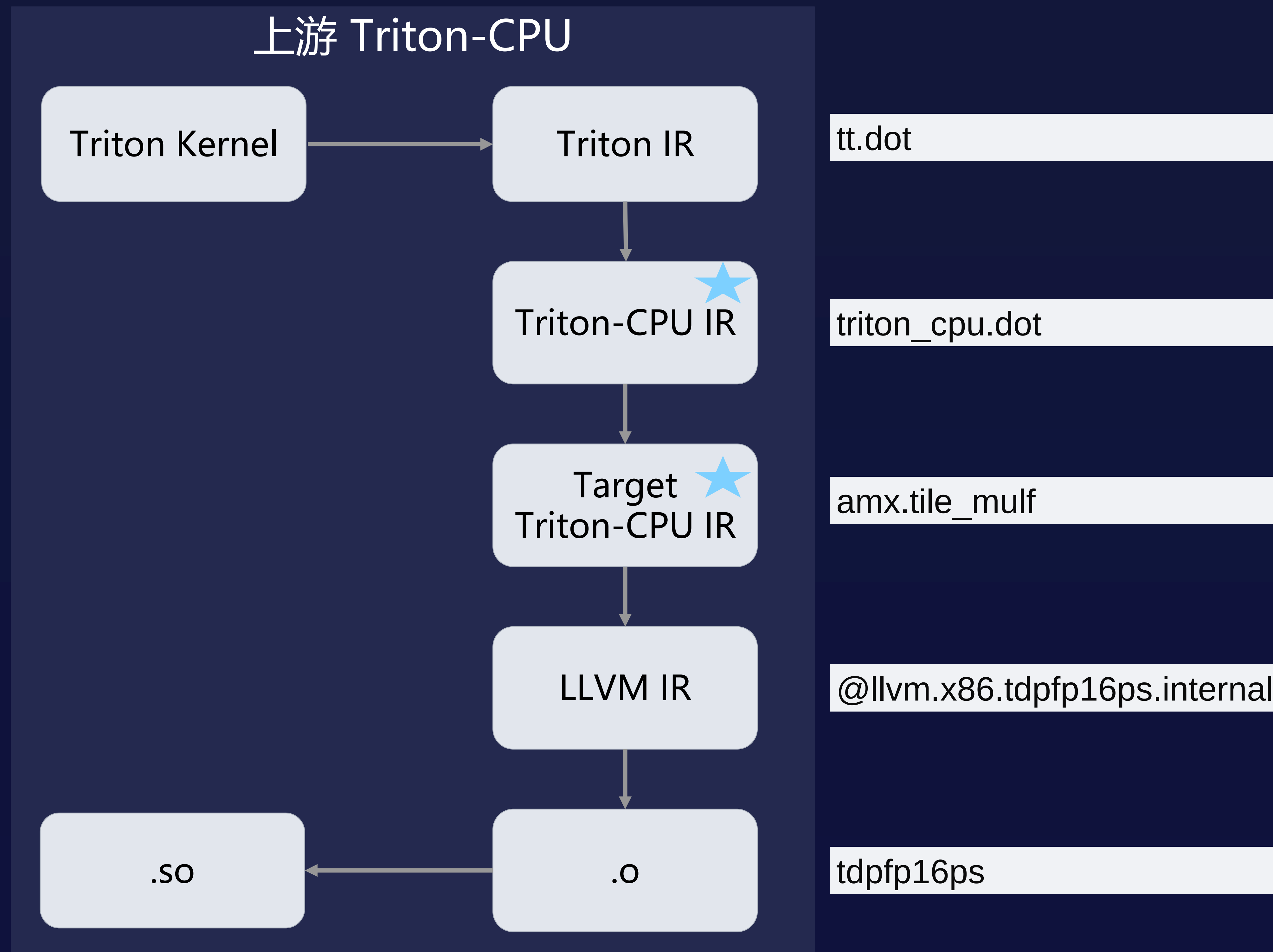
TritonKernels

AI4S

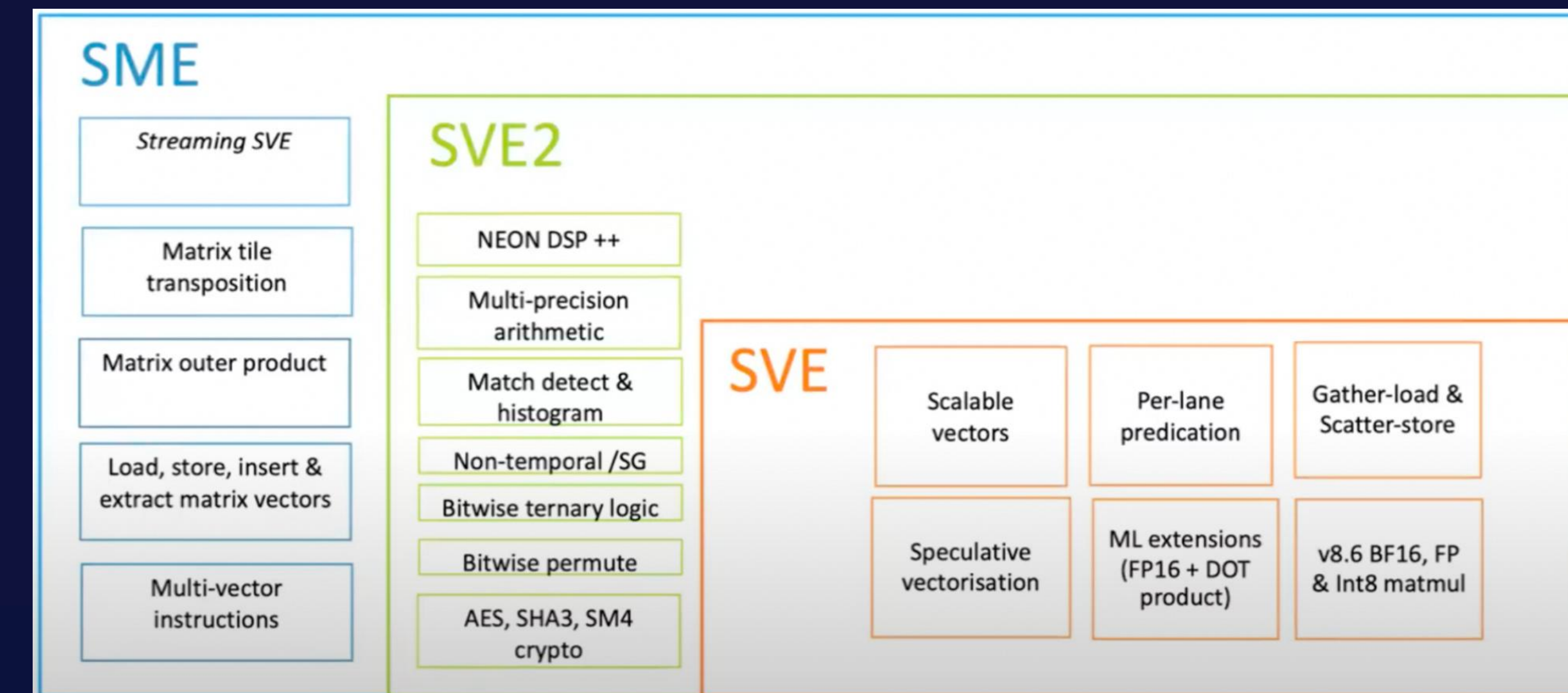
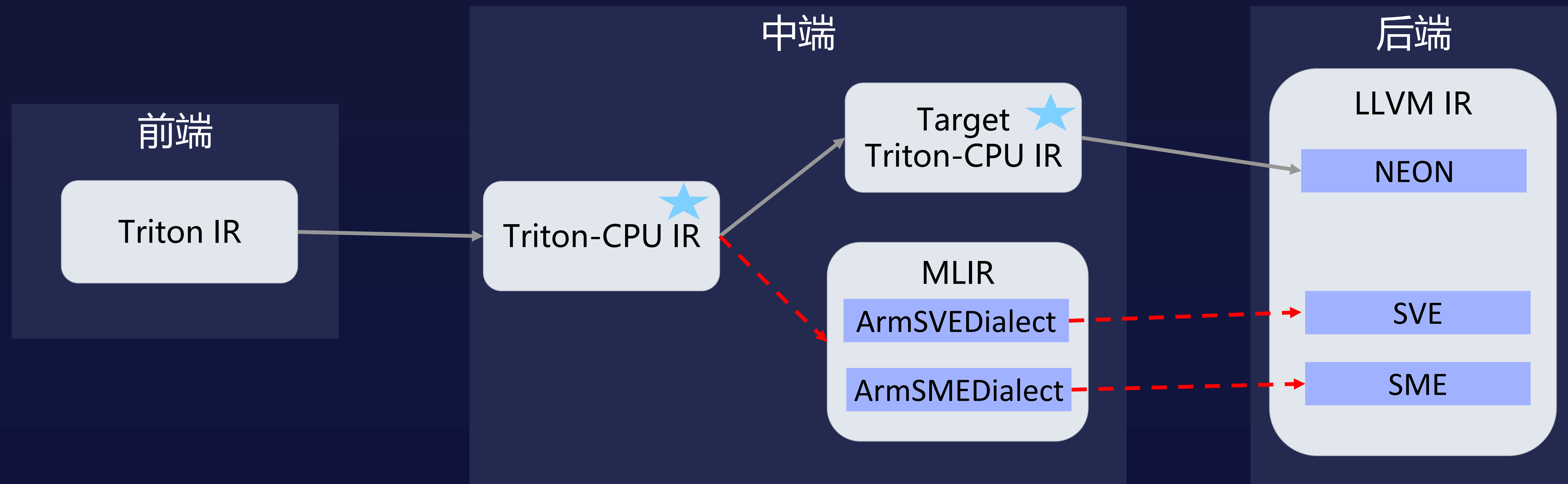
AI 推理

.....

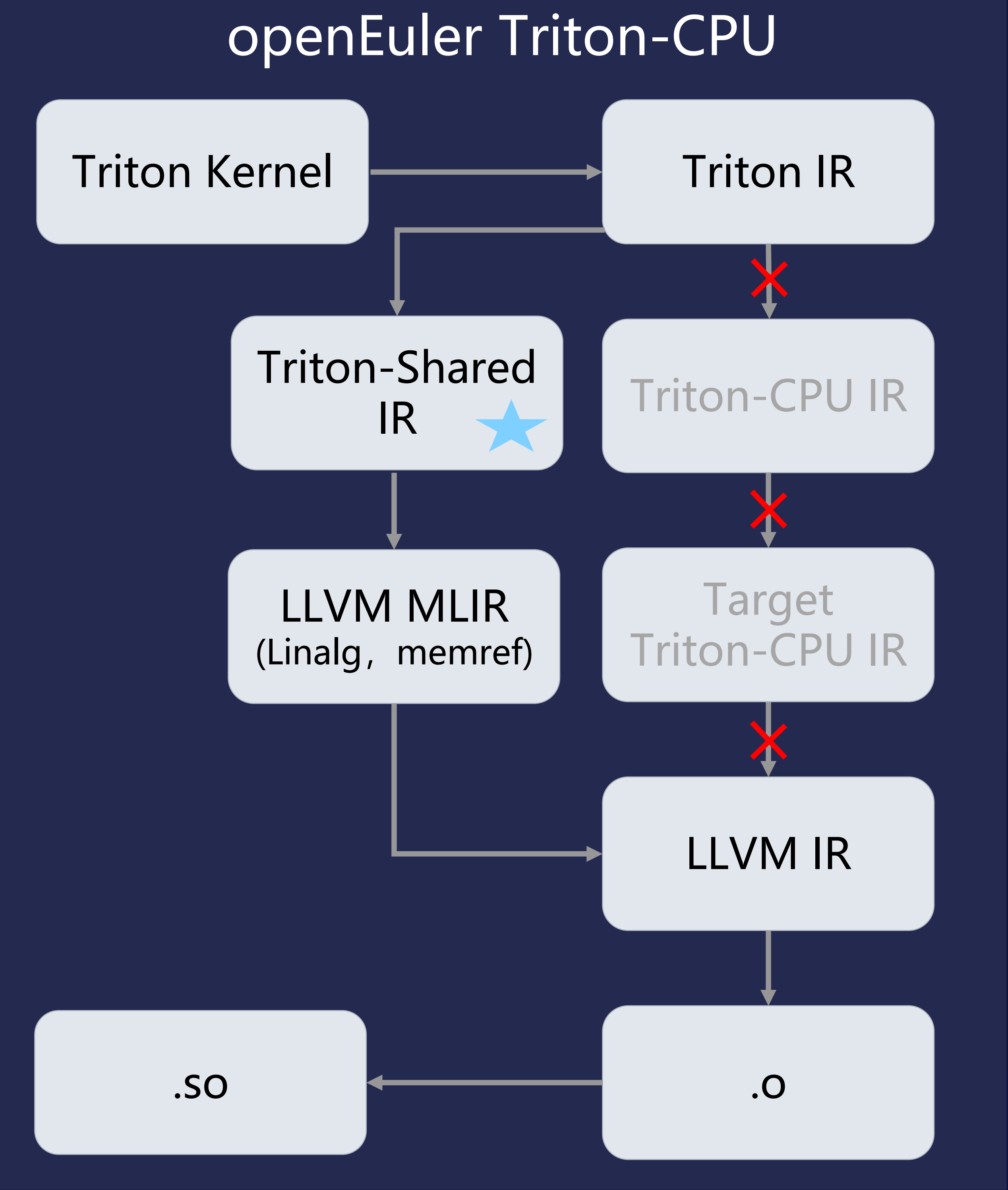
Triton-CPU上游社区方案



Triton-CPU对接AArch64后端openEuler方案



依赖Triton-Shared的方案



- Triton-Shared:
- 微软主导的适配Triton编译器的多后端中间层
 - 将Triton IR lowering成MLIR标准方言

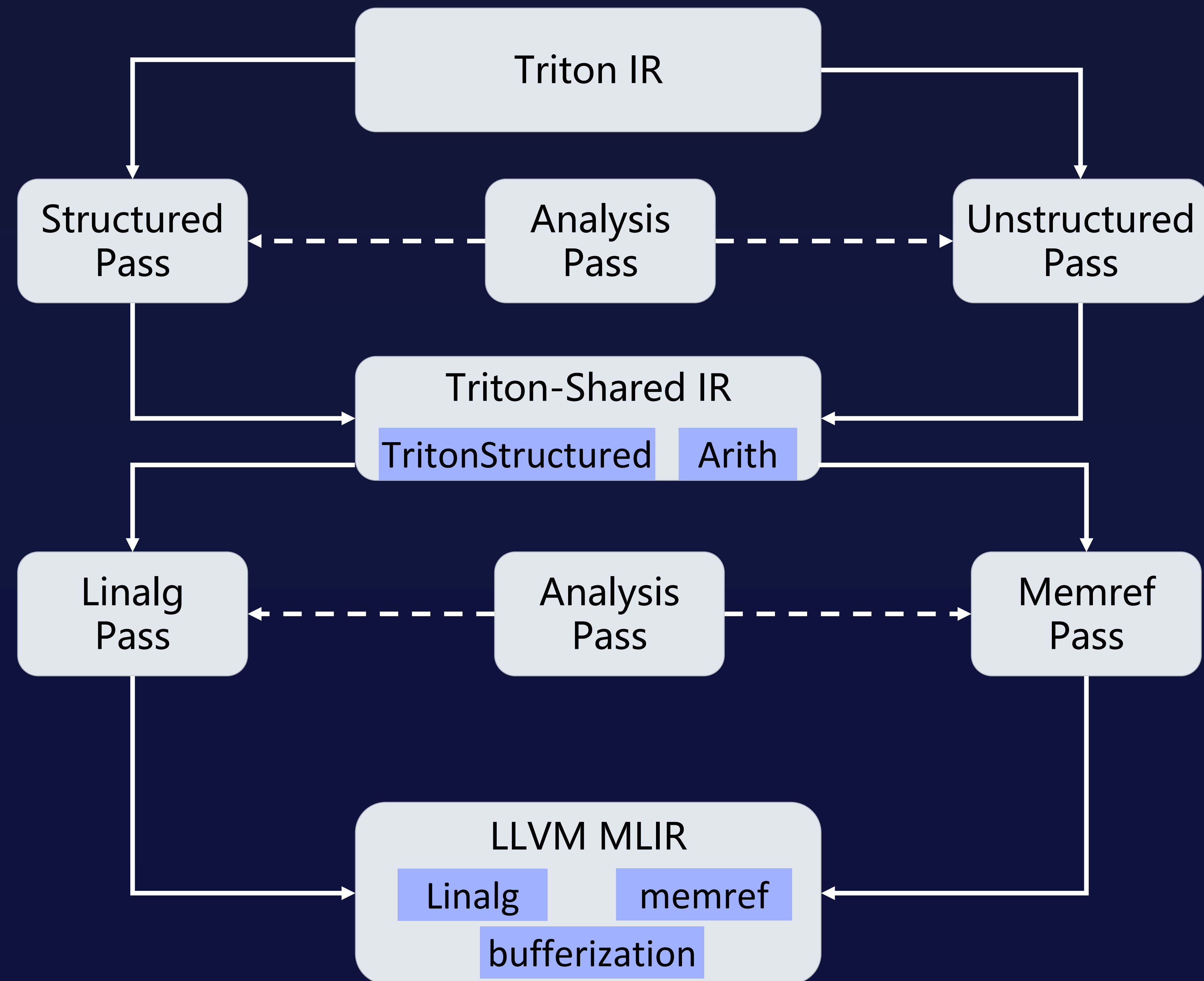
- 开发灵活性高：依赖标准的MLIR，具有良好扩展性
- 代码耦合度低：更容易适配ARM/RISCV等架构
- 性能优化方便：依赖MLIR提供的优化，更快进行性能调优

Triton-Shared的原理

```
tt.func public @kernel(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32}, %arg1: !tt.ptr<i32> {tt.divisibility = 16 : i32}) attributes {noinline = false} {
  %0 = tt.make_range {end = 8 : i32, start = 0 : i32} : tensor<8xi32>
  %1 = tt.splat %arg1 : !tt.ptr<i32> -> tensor<8x!tt.ptr<i32>>
  %2 = tt.addptr %1, %0 : tensor<8x!tt.ptr<i32>>, tensor<8xi32>
  %3 = tt.load %2 : tensor<8x!tt.ptr<i32>>
  %4 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<8x!tt.ptr<f32>>
  %5 = tt.addptr %4, %3 : tensor<8x!tt.ptr<f32>>, tensor<8xi32>
  %6 = tt.load %5 : tensor<8x!tt.ptr<f32>>
  %7 = tt.splat %arg2 : !tt.ptr<f32> -> tensor<8x!tt.ptr<f32>>
  %8 = tt.addptr %7, %0 : tensor<8x!tt.ptr<f32>>, tensor<8xi32>
  tt.store %8, %6 : tensor<8x!tt.ptr<f32>>
  tt.return
}
```

```
func.func @kernel(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32}, %arg3: i32, %arg4: i32, %arg5: i32, %arg6: i32) {
  %0 = tts.make_tptr %arg1 to sizes: [8], stride: [1]
  %1 = "tts.load"(%0) <{operandSegmentSizes = [1]}> : tensor<8xf32>
  %2 = tts.gather %arg0[%1] : (<f32>, tensor<8xf32>) -> tensor<8xf32>
  %3 = tts.make_tptr %arg2 to sizes: [8], stride: [1]
  "tts.store"(%3, %2) <{static_mask_dims = [0]}>
  return
}
```

```
func.func @kernel(%arg0: memref<*xf32> {tt.divisibility = 16 : i32}, %arg3: i32, %arg4: i32, %arg5: i32, %arg6: i32) {
  %reinterpret_cast = memref.reinterpret_cast %arg1 to <?xf32>
  %alloc = memref.alloc() : memref<8xi32>
  memref.copy %reinterpret_cast, %alloc : memref<8xi32>, memref<8xi32>
  %0 = bufferization.to_tensor %alloc restrict writable : tensor<8xf32>
  %cast = memref.cast %arg0 : memref<*xf32> to memref<?xf32>
  %1 = bufferization.to_tensor %cast restrict : memref<?xf32>
  %2 = tensor.empty() : tensor<8xf32>
  %3 = linalg.generic {indexing_maps = [#map, #map], iters = [^bb0(%in: i32, %out: f32)]} {
    %4 = arith.index_cast %in : i32 to index
    %extracted = tensor.extract %1[%4] : tensor<?xf32>
    linalg.yield %extracted : f32
  } -> tensor<8xf32>
  %reinterpret_cast_0 = memref.reinterpret_cast %arg2 to <?xf32>
  bufferization.materialize_in_destination %3 in %reinterpret_cast_0
  return
}
```



Triton-Shared的限制及解决方案

算子未处理:

原子操作



- AtomicRMW lowering
- AtomicCAS lowering
- 浮点原子操作处理

数学库函数



- Sleef 库支持
- Math dialect支持: 1, 2

其他算子

- Tptr: IntToPtrOp 和 PtrToIntOp lowering
- Triton: HistogramOp lowering

Triton-Shared的限制及解决方案

F8类型处理：

- llvm-project: 增加F8 -> I8转换Pass
- Triton-CPU: 应用F8->I8转换Pass

I1类型load/store操作处理：

- llvm-project: 增加I1 -> I8转换Pass
- Triton-CPU: 应用I1->I8转换Pass

Non row-major tensor处理：

- 支持non row-major tensor

Argmin/Argmax int类型处理：

- 支持int类型

当前Triton-CPU的状态及未来规划

LLVM 仓: https://atomgit.com/openeuler/llvm-project/tree/dev_20.1.8
Triton-CPU 仓: <https://atomgit.com/openeuler/triton-cpu>

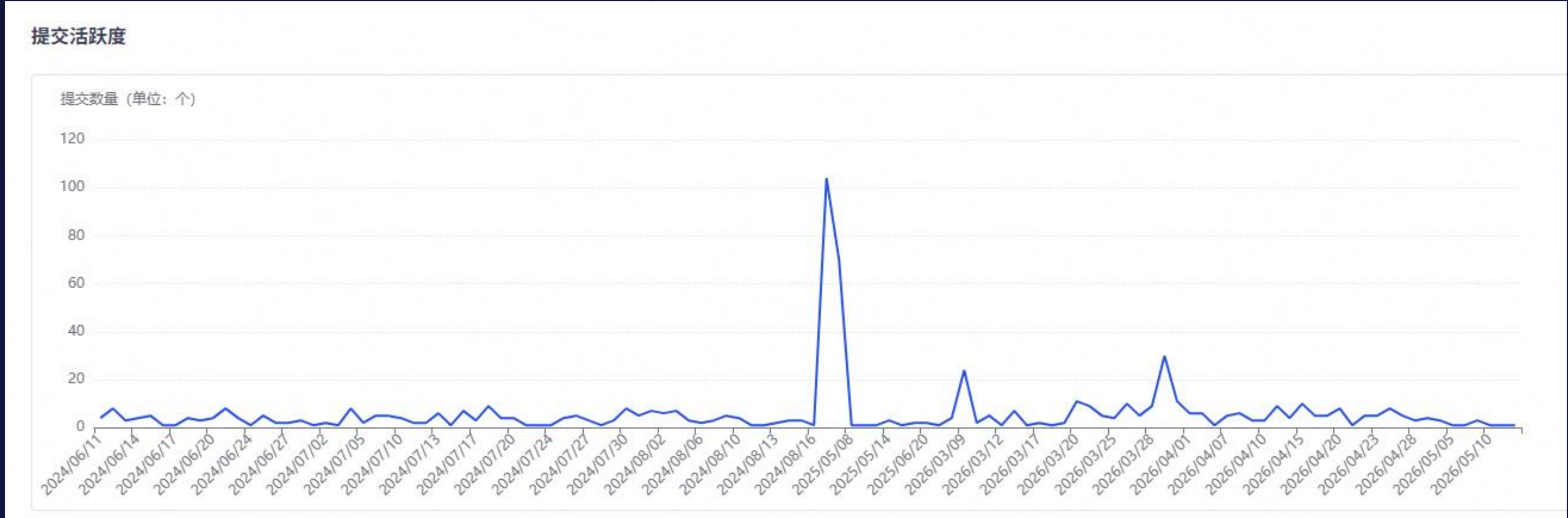
当前进展：（华为鲲鹏服务器）

- Triton-CPU自有语言特性测试全量通过
- FlagGems 4.2（216个算子）测试全量通过

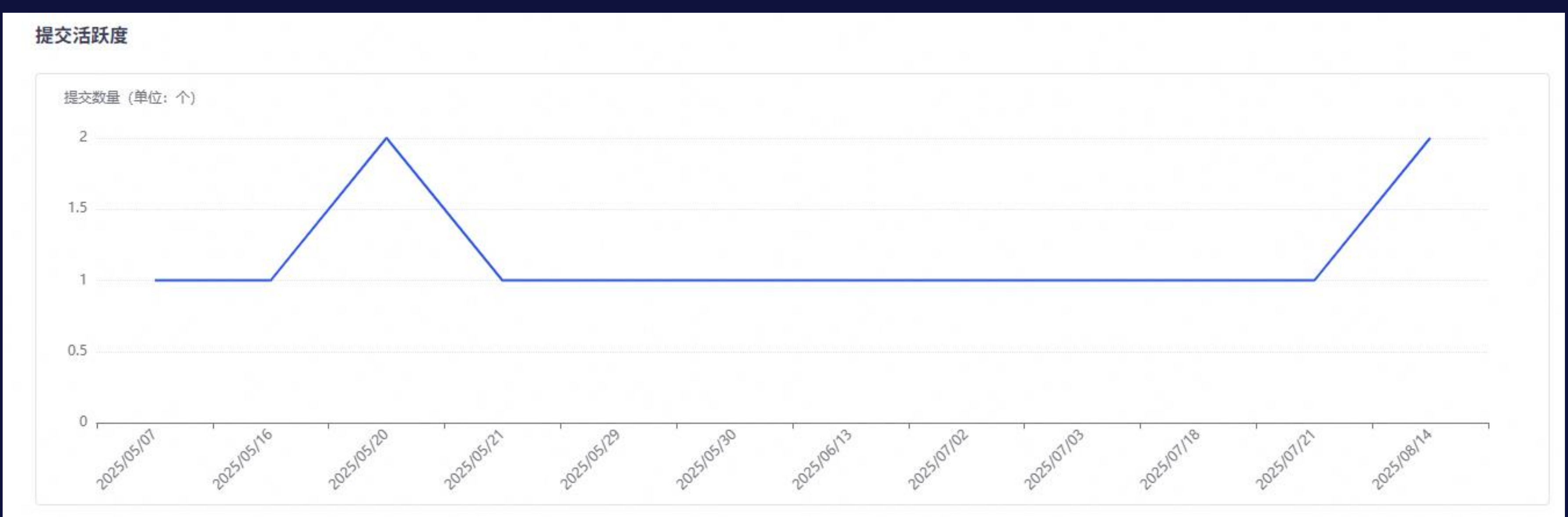
未来规划：

- SVE/SME优化流水线调优
- 24.03 SP4发布第一个SVE/SME支持版本
- 24.03 SP5发布高效SVE/SME版本

Triton-CPU源码仓



Triton-CPU制品仓





开放原子开源基金会
OPENATOM FOUNDATION



OpenEuler

Thanks!

